OS Lab

Skill-16

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50
#define MAX_JOBS 20

// ==============================
// JOB STRUCTURE
// ==============================

typedef struct
{
   int job_id;
   pid_t pid;
   char command[MAX_INPUT];
   char state[20];
} Job;

// Job table
Job jobs[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;


// ==============================
```

```
// ADD JOB
// ==============================

void add_job(pid_t pid, char *command)
{
   if (job_count >= MAX_JOBS)
   {
      printf("Job table is full\n");
      return;
   }

   jobs[job_count].job_id = next_job_id++;
   jobs[job_count].pid = pid;

   strcpy(jobs[job_count].command, command);

   strcpy(jobs[job_count].state, "Running");

   printf("[%d] %d Running    %s\n",
         jobs[job_count].job_id,
         jobs[job_count].pid,
         jobs[job_count].command);

   job_count++;
}


// ==============================
// REMOVE JOB
// ==============================

void remove_job(int index)
{
   for (int i = index; i < job_count - 1; i++)
   {
      jobs[i] = jobs[i + 1];
   }

   job_count--;
}


// ==============================
// UPDATE JOB STATUS
```

```
// ================================

void update_jobs()
{
   for (int i = 0; i < job_count; i++)
   {
      int status;

      pid_t result =
         waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
         );

      // Process finished
      if (result > 0)
      {
         strcpy(
            jobs[i].state,
            "Completed"
         );

         printf(
            "\n[%d] Done    %s\n",
            jobs[i].job_id,
            jobs[i].command
         );

         // Remove completed job
         remove_job(i);

         i--;
      }
   }
}


// ================================
// DISPLAY JOBS
// ================================

void show_jobs()
{
```

```
    update_jobs();

    if (job_count == 0)
    {
        printf("No active jobs\n");
        return;
    }

    printf("\nJob ID\tPID\tState\t\tCommand\n");
    printf("------------------------------------------\n");

    for (int i = 0; i < job_count; i++)
    {
        printf(
            "[%d]\t%d\t%s\t\t%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].state,
            jobs[i].command
        );
    }

    printf("\n");
}


// ==============================
// EXECUTE COMMAND
// ==============================

void execute_command(
    char *input,
    int background
)
{
    char input_copy[MAX_INPUT];

    strcpy(input_copy, input);

    char *args[MAX_ARGS];

    int argc = 0;

    char *token =
```

```
    strtok(input, " \t\n");

while (
    token != NULL &&
    argc < MAX_ARGS - 1
)
{
    args[argc++] = token;

    token = strtok(NULL, " \t\n");
}

args[argc] = NULL;

if (argc == 0)
    return;


// ================================
// FORK PROCESS
// ================================

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    return;
}


// ================================
// CHILD
// ================================

if (pid == 0)
{
    execvp(args[0], args);

    perror("execvp");

    exit(EXIT_FAILURE);
}
```

```
    // ================================
    // BACKGROUND PROCESS
    // ================================

    if (background)
    {
        add_job(pid, input_copy);

        /*
         * Do NOT wait here.
         * This makes the process non-blocking.
         */

        return;
    }


    // ================================
    // FOREGROUND PROCESS
    // ================================

    waitpid(pid, NULL, 0);
}


// ================================
// MAIN
// ================================

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        // Check background jobs
        update_jobs();

        printf("background$ ");

        fflush(stdout);

        if (
```

```
    fgets(
        input,
        sizeof(input),
        stdin
    ) == NULL
)
{
    break;
}

input[
    strcspn(input, "\n")
] = '\0';


if (strlen(input) == 0)
    continue;


// ================================
// EXIT
// ================================

if (
    strcmp(
        input,
        "exit"
    ) == 0
)
{
    break;
}


// ================================
// JOBS
// ================================

if (
    strcmp(
        input,
        "jobs"
    ) == 0
)
```

```
    {
        show_jobs();
        continue;
    }


    // ================================
    // DETECT &
    // ================================

    int background = 0;

    int length = strlen(input);

    if (
        length > 0 &&
        input[length - 1] == '&'
    )
    {
        background = 1;

        // Remove &
        input[length - 1] = '\0';

        // Remove trailing spaces
        length = strlen(input);

        while (
            length > 0 &&
            input[length - 1] == ' '
        )
        {
            input[length - 1] = '\0';
            length--;
        }
    }


    // Execute command
    execute_command(
        input,
        background
    );
}
```

```
    return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~/OSLab$ nano background.c
lahari@DESKTOP-AIC33F7:~/OSLab$ gcc background.c -o background
lahari@DESKTOP-AIC33F7:~/OSLab$ ./background
background$ sleep 10 &
[1] 631 Running    sleep 10
background$ jobs

[1] Done    sleep 10
No active jobs
background$ sleep 20 &
[2] 632 Running    sleep 20
background$ jobs

Job ID  PID     State           Command
-----------------------------------------
[2]     632     Running         sleep 20

background$ sleep 3 &
[3] 633 Running    sleep 3
background$ jobs

[2] Done    sleep 20

[3] Done    sleep 3
No active jobs
background$ sleep 5
background$ sleep 5 &
[4] 639 Running    sleep 5
background$
```